Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim **Ön Bilgi:** Hafta 2–3'ten taşıdıklarınız: `araclar::IntTampon` — ham `int* veri` + `int boyut`; kurucu `new[]` ile ayırır, yıkıcı `delete[]` ile bırakır (RAII'ye ilk bakış). Sınıf ve kapsülleme, `public/private`, kurucu/yıkıcının biçimsel terimleri, üye başlatma listesi ve `this`; `const` doğruluğu ve `const` referansla kopyasız salt-okunur geçiş. Ama `IntTampon`'un kopyalanınca ne olduğuna hiç bakmamıştınız — tam da bu haftanın açılış sorunu budur. Bu hafta ona kopya kurucu ile kopya atama operatörünü ekleyip Rule of 3 ile elle tamamlar, ardından akıllı işaretçilerle (Rule of 0) hiç elle yönetmeden yeniden oluşturursunuz.

» Sığ Kopya vs Derin Kopya

Sığ Kopya (Tehlike)

Kullanıcı hiçbir şey yazmazsa derleyici kopya kurucu/atamayı *üye üye* üretir: işaretçi üyede adresin kendisi kopyalanır, işaret edilen veri değil. Sonuç: iki nesne aynı bloğu paylaşır (paylaşılan durum) ve kapsam sonunda aynı adres iki kez `delete[]` edilir (çifte delete).

Derin Kopya (Çözüm)

Kopya kurucu/atama kendi `new[]`'ini yapıp elemanları tek tek kopyalar; iki nesne artık bağımsız birer kaynağa sahiptir, biri değişse öteki etkilenmez. Genel iskelet: `T(const T& diger)` / `T& operator=(const T& diger)`.

» `IntTampon`: Kopya Kurucu & Atama

```
IntTampon(const IntTampon& diger) {
   boyut = diger.boyut;
   veri = new int[boyut];
   for (int i = 0; i < boyut; ++i)
      veri[i] = diger.veri[i];
}
IntTampon& operator=(const IntTampon& diger) {
   if (this == &diger) return *this; // oz-atama
   delete[] veri; // eski kaynagi birak
   boyut = diger.boyut;
   veri = new int[boyut];
   for (int i = 0; i < boyut; ++i)
      veri[i] = diger.veri[i];
   return *this;
}
```

Kontrolsüz: `delete[] veri;` sonrası `diger.veri` da (aynı adres) geçersizleşir — döngü geçersiz bellekten okur.

» Rule of 3, `=default`/`=delete`

Rule of 3

Yıkıcı/kopya kurucu/kopya atamadan biri elle yazılırsa genelde *üçü de* yazılmalı — üçü de aynı soruyu (kaynağın sahibi bu sınıf mı?) yanıtlar; biri unutulursa sızıntı/çifte delete riski doğar.

`=default` / `=delete`

`Sayac(const Sayac&) = default;` — kaynak yok, memberwise kopya zaten doğru, niyet açık. `TekilKart(const TekilKart&) = delete;` — kopyalama tamamen yasak, deneme derleme hatası verir.

Rule of 0 / Rule of 5 (kısa not)

Rule of 0: kaynağı hazır üyelere (`std::string`, akıllı işaretçi, STL) devredin — hiçbir özel fonksiyon elle yazılmaz. Rule of 5: taşıma kurucusu + taşıma atama da eklenince beşli (yalnız isim düzeyinde, bu hafta derinleşilmez).

Taşıma (*move*) — kısa değini

`std::move(x)`, `x`'i rvalue referansı olarak işaretler. Taşıma kurucusu elle yazılmadıysa (bu hafta yazılmaz) `std::move` yine derlenir ama kopya kurucuya düşer — güvenli, verimsiz.

» Akıllı İşaretçiler

	`unique_ptr`	`shared_ptr`
Sahiplik	tekil	paylaşılan
Kopyalanır mı	hayır	evet (sayaç++)
Taşınır mı	evet (`move`)	evet
Oluşturma	`make_unique`	`make_shared`
Sayaç	yok	`use_count()`

`AkilliTampon` (Rule of 0 Örneği)

```
class AkilliTampon {
public:
   explicit AkilliTampon(int n)
     : veri_(std::make_unique<int[]>(n)),
       boyut_(n) {}
private:
   std::unique_ptr<int[]> veri_;
   int boyut_;
};
```

Kullanıcı tanımlı yıkıcı/kopya kurucu/kopya atama YOK — `unique_ptr` üyesi kopyalanamaz olduğu için sınıf kendiliğinden kopyalanamaz olur.

» `IntBagliListe`: Düğüm ve Sahiplik

Düğüm + Tek Üye

```
struct Dugum {
   int deger;
   std::unique_ptr<Dugum> sonraki;
};
std::unique_ptr<Dugum> bas_; // tek uye
```

Her düğümü yalnız *bir* önceki sahiplenir (başsa liste, değilse önceki düğüm) — `unique_ptr` ile birebir uyum. Yıkıcı elle YAZILMAZ (Rule of 0): `bas_` yok olurken zincir ardışık yok olur.

`ekleBasa` / `yazdir`

```
void ekleBasa(int deger) {
   auto n = std::make_unique<Dugum>();
   n->deger = deger;
   n->sonraki = std::move(bas_);
   bas_ = std::move(n);
}
```

Sıra zorunlu: önce eski `bas_` yeni düğümün altına taşınır, sonra yeni düğüm başa taşınır — zincir hiç kopmaz. `yazdir() const` sahiplik almayan bir gözlemci ham işaretçiyle (`const Dugum*`, `.get()`) gezinir.

» Sık Yapılan Hatalar ve Seçimler

Ortak Hata / Tuzak

- Kaynak tutan sınıfta kopya kurucu/atamayı hiç yazmamak (derleyicinin sığ kopyasına güvenmek) → paylaşılan durum + çifte `delete`.
- Kopya atamada öz-atama kontrolünü (`if (this == &diger)`) atlamak → `delete[] veri;` sonrası geçersiz `diger.veri`'den okumaya çalışmak.
- Yalnız birini (ör. yıkıcı) elle yazıp kopya kurucu/atamayı varsayılana bırakmak — Rule of 3'ü yarım uygulamak.
- Tek sahip yeterliyken `shared_ptr` kullanmak: gereksiz sayaç maliyeti; ihtiyaç yoksa sahipliği `unique_ptr`'da TUTUN.

Ne Zaman Hangisi?

- Varsayılan tercih: `unique_ptr` — en ucuz (sayaç yok), en açık (tek sahip).
- `shared_ptr`: yalnızca birden çok sahip *aynı anda* kaynağı yaşatmalıysa (sayaç maliyetine değer).
- `weak_ptr` bu hafta KAPSAM DIŞI (döngüsel referans, ileri konu).

5 • İzleme **Nerede Kullanılır:** Bir sınıf ham işaretçi/kaynak (bellek, dosya tanıtıcısı) tuttuğunda: kopyalanırken kaynağı paylaşmak mı yeni bir kaynak ayırmak mı istediğinize karar verin — paylaşım tehlikeliyse kendi kopya kurucu + kopya atamanızı (derin kopya, öz-atama kontrolü) yazın ve üçlüyü Rule of 3 ile birlikte tamamlayın. Kaynak tutmayan küçük bir sınıfta üye üye kopya zaten doğruysa niyeti `= default` ile açın; bir tip asla kopyalanmamalıysa `= delete` ile kopyayı derlemede yasaklayın. Elle yönetim sorumluluğunu bütünüyle bırakmak istediğinizde kaynağı akıllı işaretçi üyeye devredip (Rule of 0) hiçbir özel fonksiyon yazmayın: tek sahip yeterliyse `unique_ptr`, kaynağı birden çok sahip aynı anda yaşatmalıysa `shared_ptr` seçin — varsayılan tercihiniz `unique_ptr` olsun. Düğümleri birbirine bağlayan bir yapı kurarken (bağlı liste, ağaç) her düğümü bir öncekine `unique_ptr` ile sahiplendirin, gezinmeyi `.get()` ile alınan sahiplenmeyen ham işaretçiyle yapın; bu desen gelecek haftalarda `Kitap` listelerine ve şablonlu `Liste<T>`'ye aynen taşınır.